

REPORT DI ANALISI E OTTIMIZZAZIONE DEL CODICE

(HANDS-ON#4)

Data di completamento: 18 Giugno 2026
Componenti Analizzati: Cartella `quality` (Moduli core: `auth.c` , `main.c`)
Strumenti Software: SciTools Understand™ (Rule Checker MISRA C 2012), GCC Compiler, GCOV, LCOV

1. OBIETTIVI DELL'ATTIVITÀ

L'obiettivo di questo laboratorio è analizzare staticamente e dinamicamente il codice sorgente per identificare violazioni delle regole di programmazione sicura, misurare le metriche strutturali e calcolare la code coverage, applicando successivamente un processo di refactoring mirato a migliorare la qualità complessiva del software.

Gli obiettivi quantitativi prefissati sono:

- Risolvere almeno 50 violazioni delle linee guida **MISRA C 2012**, includendo tutte le istanze di codice irraggiungibile (*unreachable code*).
- Abbattere la complessità ciclotomatica delle singole funzioni.

2. ANALISI STATICA DELLE VIOLAZIONI (MISRA C 2012)

L'ispezione iniziale eseguita mediante il modulo *CodeCheck* di SciTools Understand ha evidenziato un totale di **685 violazioni complessive** distribuite all'interno dei costrutti logici dei file analizzati.

In seguito alle sessioni di refactoring strutturale, il numero di non-conformità è sceso a **369 violazioni complessive**, registrando un decremento netto di **316 violazioni risolte**.

Interventi di Refactoring Applicati:

- **Rimozione del Codice Morto (*Dead-Code*):** Eliminazione dei rami condizionali `if-else` e delle istruzioni localizzate dopo i costrutti di interruzione anticipata (`return`) che risultavano strutturalmente irraggiungibili.
- **Semplificazione Sintattica:** Sostituzione dell'operatore ternario (`? :`) con strutture condizionali esplicite per aumentare la leggibilità e rispettare i vincoli tipologici MISRA.
- **Ottimizzazione delle Risorse:** Eliminazione delle variabili e dei token dichiarati ma mai interrogati all'interno dei flussi operativi.

3. ANALISI DELLE METRICHE STRUTTURALI (PRE VS POST MODIFICA)

I dati estratti tramite i report automatici di SciTools Understand mostrano l'efficacia del refactoring sia sul contenimento della complessità algoritmica sia sull'arricchimento della documentazione (rapporto commenti/codice).

Tabella Comparativa delle Metriche Generali

Name	Kind	Cyclomatic (Pre)	Cyclomatic (Post)	CountLineCode (Pre)	CountLineCode (Post)	RatioCommentToCode (Pre → Post)
<code>auth.c</code>	File	-	-	159	193	0,01 → 0,13
<code>auth_add</code>	Function	7	7	20	27	0,00 → 0,19

<code>auth_change_password</code>	Function	2	2	5	7	0,00 → 0,14
<code>auth_check</code>	Function	9	9	20	24	0,00 → 0,13
<code>auth_delete</code>	Function	2	2	5	7	0,00 → 0,14
<code>auth_init</code>	Function	3	3	10	12	0,00 → 0,08
<code>auth_rename_user</code>	Function	4	4	13	15	0,00 → 0,07
<code>cancella_utente</code>	Function	3	3	14	14	0,14 → 0,14
<code>crea_utente</code>	Function	3	2	20	10	0,10 → 0,60
<code>ct_str_eq</code>	Static Function	5	5	10	10	0,00 → 0,00
<code>effettua_login</code>	Function	4	3	19	13	0,00 → 0,23
<code>find_user</code>	Static Function	5	5	20	22	0,00 → 0,23
main.c	File	-	-	284	213	0,07 → 0,18
<code>main</code>	Function	15	11	51	32	0,04 → 0,22
<code>modifica_utente</code>	Function	71	55	143	132	0,00 → 0,11
<code>read_line</code>	Function	3	3	10	12	0,00 → 0,00
<code>rewrite_db_skip_or_replace</code>	Static Function	16	16	51	62	0,02 → 0,11
<code>trim_nl</code>	Static Function	2	2	4	6	0,00 → 0,17

Nota sulle funzioni rimosse: Le sotto-funzioni fittizie di test presenti nello stato iniziale (`demo_comment_mix` , `demo_ub` , `demo_unspecified` , `f_side` , `g_side`) sono state completamente epurate dalla base di codice nel corso del refactoring strutturale per azzerare il codice inutilizzato.

Analisi dei Risultati Metrici:

- **Abbattimento Complessità (Cyclomatic):** La funzione critica `modifica_utente` (in `main.c`) è stata parzialmente ristrutturata riducendo il valore dell'indagine Cyclomatic da un livello iper-critico di **71** a **55**. Allo stesso modo, le sotto-funzioni `crea_utente` (da 3 a 2), `effettua_login` (da 4 a 3) e lo stesso entry point `main` (da 15 a 11) hanno registrato contrazioni virtuose nell'indice di complessità.
- **Linee di Codice (CountLineCode):** Il perimetro complessivo di `main.c` ha ridotto le linee totali da **284** a **213** grazie alla pulizia del codice morto. Al contrario, `auth.c` ha registrato un aumento fisiologico (da 159 a 193) dovuto all'espansione dei costrutti ternari in blocchi `if-else` estesi.
- **Integrazione Commenti (RatioCommentToCode):** Il parametro di documentazione è migliorato in modo generalizzato (ad esempio su `auth.c` è passato da 0,01 a 0,13 e su `crea_utente` da 0,10 a 0,60), garantendo la piena manutenibilità futura del codice.

4. ANALISI DINAMICA E CODE COVERAGE (GCOV/LCOV)

L'analisi di copertura è stata condotta compilando l'applicazione tramite GCC con i flag di profilazione attivi ed escludendo le ottimizzazioni della build per prevenire alterazioni o rimozioni preventive dei rami decisionali da parte del compilatore:

```
gcc *.c -std=c17 -Wall -Wextra -O0 -fprofile-arcs -ftest-coverage -o app.exe
```